# Интеграция frontend и backend

Fetch API, общий apiFetch(), JWT и ответы REST API на примере Internet Shop

# Задача занятия

- Связать существующий Express REST API с HTML-страницей Internet Shop.
- Отправлять запросы из браузера через fetch().
- После входа отправлять JWT в защищённые API-маршруты.
- Цель — обмен данными, а не построение страниц и меню: это тема лекции №6.

# Чему научимся

- Отправлять GET и POST с JSON.
- Различать успешный HTTP-ответ, ошибку API и сетевую ошибку.
- Создавать единый модуль apiFetch() и не повторять fetch() повсюду.
- Сохранять JWT после login и передавать Authorization: Bearer.
- Выводить результат запроса или понятную ошибку.

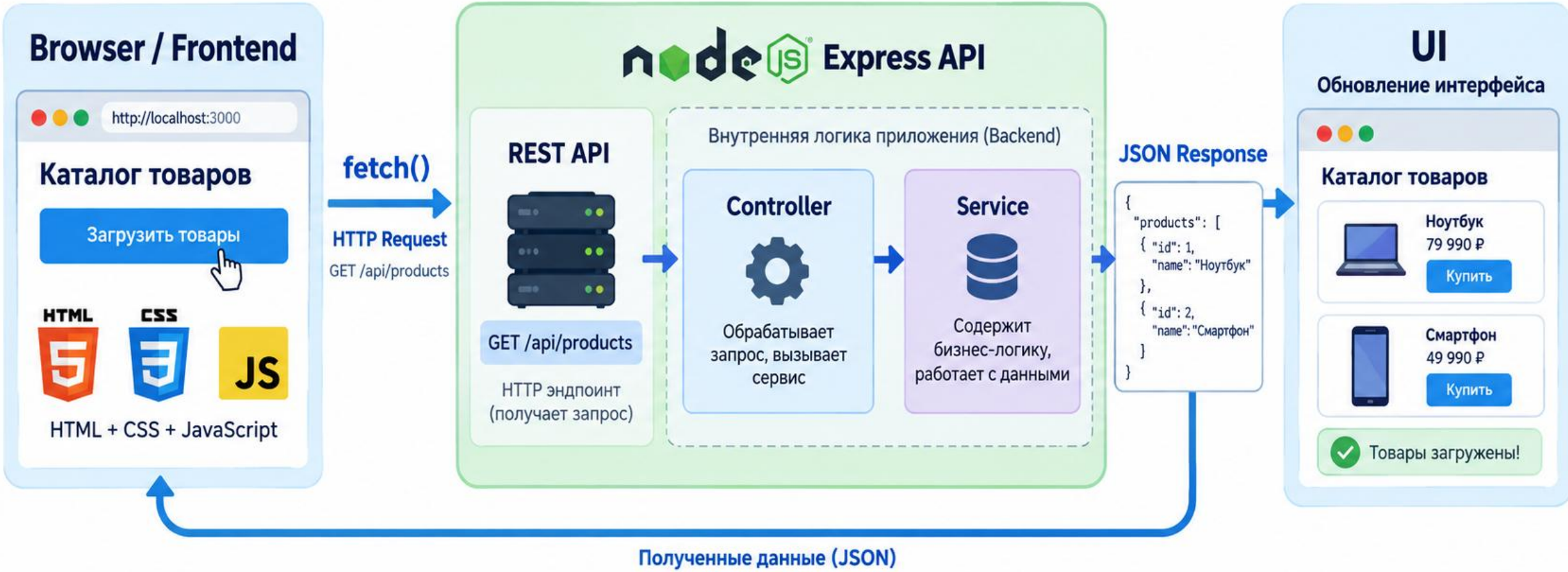
# Контракт Internet Shop API

| Действие | HTTP | URL | Результат |
|---|---|---|---|
| Вход | POST | /api/auth/login | user + accessToken |
| Каталог | GET | /api/products | Список товаров |
| Текущий пользователь | GET | /api/auth/me | Данные пользователя |
| Мои заказы* | GET | /api/orders/my | Заказы пользователя |

# Как браузер общается с API

- Frontend отправляет HTTP-запрос: method, URL, headers и иногда body.
- Express обрабатывает маршрут и возвращает HTTP status и JSON.
- Frontend читает ответ и обновляет страницу.
- API-контракт должен совпадать в серверном и клиентском коде.
- Примечание к предыдущему слайду: Маршрут заказов используется, только если он реализован в backend.

# Frontend → HTTP Request → Express API → JSON Response → UI



# Простейший GET

```javascript
const response = await fetch('/api/products');
const products = await response.json();
console.log(products);
```

### Что важно

- GET используется по умолчанию.
- response.json() читает и преобразует JSON-ответ.
- Работает с Express, обслуживающим static/ по адресу localhost:3000.

# fetch() и ошибки

- HTTP 404 или 500 не обязательно вызывают исключение в fetch().
- response.ok равен true для статусов от 200 до 299.
- Ошибку API определяем по response.ok / response.status.
- Сетевая ошибка обычно приводит к исключению и обработке через catch.

# fetch(): network error vs HTTP error vs successful JSON

Один и тот же код **fetch()** может привести к разным результатам. Важно проверять **response.ok** и обрабатывать ошибки.

**Internet Shop**
Learn. Build. Deploy.

**Frontend (Browser)**



fetch('/api/orders')

HTTP request

```
try {
  const response = await fetch('/api/orders', { headers: { 'Authorization': 'Bearer ' + token } });
  // обработка в зависимости от результата
} catch (error) {
  // срабатывает только при сетевой ошибке
}
```

Запрос к API



**Express API**
/api/orders

## 1. Network Error

Сетевая ошибка (нет соединения)




**API недоступен**
- Сервер не запущен
- Нет интернета
- Неверный домен
- Таймаут подключения

```
try {
  await fetch('http://localhost:3000/api/orders');
} catch (error) {
  console.error('Network error:', error);
}
```

**Результат**

**fetch() выбрасывает исключение (exception)**

- TypeError: Failed to fetch
- TypeError: NetworkError when attempting to fetch resource
- и другие ошибки сети

## 2. HTTP Error (401, 404, 500)

Сервер ответил с кодом ошибки




**Примеры:**
- **401 Unauthorized** (нет токена / неверный токен)
- **404 Not Found** (ресурс не найден)
- **500 Internal Server Error** (ошибка сервера)

```
const response = await fetch('/api/orders');
console.log(response.ok);      // false
console.log(response.status);  // 401, 404, 500
const text = await response.text();
```

**Результат**

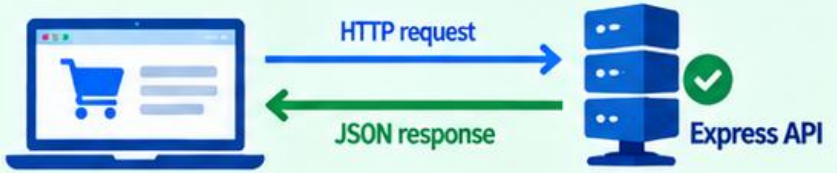
**fetch() не выбрасывает исключение.**

Возвращается объект Response:
- response.ok = false
- response.status = 401 (или 404, 500 и т.д.)
- можно прочитать текст ошибки: await response.text()

```
Response {
  ok: false,
  status: 401,
  statusText: "Unauthorized",
  ...
}
```

## 3. Successful JSON (200)

Успешный ответ с JSON




```
const response = await fetch('/api/orders');
console.log(response.ok);      // true
console.log(response.status);  // 200
const data = await response.json();
console.log(data);
```

**Результат**

**fetch() возвращает успешный Response:**
- response.ok = true
- response.status = 200
- можно вызвать await response.json() и получить данные

```
Response {
  ok: true,
  status: 200,
  statusText: "OK",
} ...
```

→

```
[
  { "id": 1, "name": "Ноутбук", "price": 499 },
  { "id": 2, "name": "Мышь", "price": 25 }
]
```

Проверяйте **response.ok** и обрабатывайте ошибки в обоих случаях: сетевые (catch) и HTTP (ok = false).

```
if (!response.ok) {
   throw new Error(`HTTP error: ${response.status}`);
}
```

Надёжное приложение корректно обрабатывает все варианты

# Проверка результата GET

```javascript
try {
  const response = await fetch('/api/products');
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  const products = await response.json();
  console.log(products);
} catch (error) {
  console.error(error.message);
}
```

**Что важно**

- Обрабатываем как сетевые ошибки, так и ответы с ошибкой HTTP.
- Без !response.ok HTTP 404 мог бы попасть в ветку успеха.

# POST с JSON: вход пользователя

```javascript
const response = await fetch('/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
});
```

### Что важно

- method задаёт POST.
- Content-Type сообщает серверу формат тела.
- JSON.stringify превращает объект JavaScript в JSON-строку.

# Зачем общий API-клиент

- Без общей функции в каждом файле повторяются URL, заголовки и проверки ошибок.
- Общий apiFetch() централизует формат JSON, JWT и обработку HTTP-ответа.
- Компонент интерфейса передаёт только маршрут и параметры запроса.

# config.js: один базовый адрес

```JAVASCRIPT
// static/config.js
export const API_URL = '/api';
```

**Что важно**

- Frontend и backend запускаются одним Express-сервером по http://localhost:3000/.
- Используем относительный путь: отдельный frontend-сервер и CORS не нужны.

# apiFetch(): минимальная версия

```javascript
import { API_URL } from './config.js';

export async function apiFetch(path, options = {}) {
  const response = await fetch(`${API_URL}${path}`,
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.status === 204 ? null : response.js
}
```

**Что важно**

- path добавляется к /api.
- Функция проверяет HTTP-статус в одном месте.
- 204 No Content не имеет тела JSON.

# Автоматическая отправка JSON

```javascript
// Фрагмент внутри apiFetch()
const response = await fetch(`${API_URL}${path}`, {
  ...options,
  headers: {
    ...(body !== undefined ? { 'Content-Type': 'appli
    ...headers
  },
  body: body !== undefined ? JSON.stringify(body) : u
});
```

**Что важно**

- body передаётся обычным JS-объектом.
- ...options подставляет method и другие настройки.
- ...headers добавляет заголовки, переданные вызывающим кодом.

# После успешного login

- Backend возвращает объект user и JWT в поле accessToken.
- На этой лабораторной токен хранится в переменной JavaScript, а не в localStorage.
- Для следующего защищённого запроса frontend передаёт JWT в Authorization header.
- После обновления страницы token в памяти исчезает — это ограничение учебной версии.

# authState.js: хранение результатов входа

```javascript
let accessToken = null;
let currentUser = null;

export function setAuth(data) {
  accessToken = data.accessToken;
  currentUser = data.user;
}
export function getAccessToken() { return accessToken
export function getCurrentUser() { return currentUser
export function clearAuth() {
  accessToken = null;
  currentUser = null;
}
```

### Что важно

- setAuth записывает ответ после login.
- getAccessToken используется общим API-клиентом.
- Подробное управление страницами по ролям будет в лекции №6.

# Bearer JWT в apiFetch()

JAVASCRIPT

```
import { getAccessToken } from './authState.js';

// Перед fetch():
const token = getAccessToken();

// Внутри объекта headers:
...(token ? { Authorization: `Bearer ${token}` } : {})
```

**Что важно**

- Если токен есть, добавляется заголовок Authorization.
- Публичные запросы работают и без токена.
- Backend проверяет JWT самостоятельно с помощью requireAuth.

# login(): один запрос — один результат

```javascript
import { apiFetch } from './api.js';
import { setAuth } from './authState.js';

export async function login(email, password) {
  const data = await apiFetch('/auth/login', {
    method: 'POST',
    body: { email, password }
  });
  setAuth(data);
  return data.user;
}
```

**Что важно**

- apiFetch отправляет JSON на backend.
- При успехе setAuth сохраняет user и accessToken.
- Функцию вызывают из обработчика HTML-формы.

# Форма входа: submit

```javascript
form.addEventListener('submit', async (event) => {
  event.preventDefault();
  loginButton.disabled = true;
  errorBox.textContent = '';
  try {
    const user = await login(emailInput.value, passwo
    welcome.textContent = `Здравствуйте, ${user.name}
  } catch (error) {
    errorBox.textContent = error.message;
  } finally {
    loginButton.disabled = false;
  }
});
```

**Что важно**

- preventDefault отменяет стандартную отправку HTML-формы.
- try показывает результат, catch — ошибку.
- finally разблокирует кнопку при любом исходе.

# APIError: сохранить HTTP status

```javascript
export class APIError extends Error {
  constructor(message, status) {
    super(message);
    this.status = status;
  }
}
```

**Что важно**

- Встроенный Error содержит message.
- APIError дополнительно содержит status — например 401 или 403.

# apiFetch(): обработка ответов

```javascript
// После fetch():
const data = response.status === 204 ? null : await r

if (response.status === 401) {
  clearAuth();
}
if (!response.ok) {
  throw new APIError(
    data?.error ?? `HTTP ${response.status}`,
    response.status
  );
}
return data;
```

**Что важно**

- 401: токен не принят; очищаем локальное состояние входа.
- 403: недостаточно прав; не выполняем logout.
- Этот пример предполагает JSON для ответов, кроме HTTP 204.

# Что делать при разных ответах

| Ситуация | Статус | Реакция frontend |
| --- | --- | --- |
| Вход выполнен | 200 | Показать имя пользователя |
| Неверные credentials | 401 | Показать ошибку входа |
| Недостаточно прав | 403 | Показать отказ без logout |
| Ресурс не найден | 404 | Показать сообщение |
| Ошибка сети | Нет HTTP-ответа | Показать ошибку соединения |

# Проверка работы через DevTools

- Откройте http://localhost:3000/ → DevTools → Network → Fetch/XHR.
- После нажатия «Войти» найдите POST /api/auth/login.
- Проверьте Request Payload, HTTP status и Response.
- Для защищённого GET проверьте заголовок Authorization: Bearer.
- Во вкладке Console посмотрите ошибки JavaScript, если запрос не появился.

# Где находятся файлы в Internet Shop

- Express продолжает обслуживать frontend из папки static/.
- В static/ размещены index.html, app.js, api.js, authState.js, config.js и styles.css.
- Отдельный web-сервер на порту 5500 и настройка CORS для этой архитектуры не нужны.
- В ЛР-5 не нужно повторно создавать backend: работа продолжается в существующем проекте.

# Практика по ЛР-5

- Создать общий apiFetch() с проверкой response.ok и поддержкой JSON.
- Подключить login form к POST /api/auth/login.
- Сохранить accessToken в памяти и добавить Bearer автоматически.
- Проверить публичный GET /api/products и один реально существующий защищённый маршрут.
- Проверить ошибки 401 и 403, не смешивая их.

# Итоги занятия

1. fetch() отправляет запросы из браузера в Express API.
2. apiFetch() централизует URL, JSON, Bearer JWT и HTTP-ошибки.
3. После login сервер возвращает user и accessToken; frontend хранит их в памяти.
4. Настоящая проверка JWT и прав всегда остаётся на backend.
5. Навигацию, protected pages и route guards изучим в лекции №6.

# Контрольные вопросы

1. Чем сетевая ошибка отличается от HTTP 404?
2. Для чего нужны response.ok и JSON.stringify()?
3. Зачем выделяют apiFetch() в отдельный модуль?
4. Откуда apiFetch() получает JWT и как передаёт его серверу?
5. Почему при 403 не следует автоматически очищать авторизацию?